

小结

在 C++ 中，内存是通过 `new` 表达式分配，通过 `delete` 表达式释放的。标准库还定义了一个 `allocator` 类来分配动态内存块。

分配动态内存的程序应负责释放它所分配的内存。内存的正确释放是非常容易出错的地方：要么内存永远不会被释放，要么在仍有指针引用它时就被释放了。新的标准库定义了智能指针类型——`shared_ptr`、`unique_ptr` 和 `weak_ptr`，可令动态内存管理更为安全。对于一块内存，当没有任何用户使用它时，智能指针会自动释放它。现代 C++ 程序应尽可能使用智能指针。

术语表

allocator 标准库类，用来分配未构造的内存。

空悬指针 (dangling pointer) 一个指针，指向曾经保存一个对象但现在已释放的内存。众所周知，空悬指针引起的程序错误非常难以调试。

delete 释放 `new` 分配的内存。`delete p` 释放对象，`delete []p` 释放 `p` 指向的数组。`p` 可以为空，或者指向 `new` 分配的内存。

释放器 (deleter) 传递给智能指针的函数，用来代替 `delete` 释放指针绑定的对象。

析构函数 (destructor) 特殊的成员函数，负责在对象离开作用域或被释放时完成清理工作。

动态分配的 (dynamically allocated) 在自由空间中分配的对象。在自由空间中分配的对象直到被显式释放或程序结束才会销毁。

自由空间 (free store) 程序可用的内存池，保存动态分配的对象。

堆 (heap) 自由空间的同义词。

new 从自由空间分配内存。`new T` 分配并构造一个类型为 `T` 的对象，并返回一个指向该对象的指针。如果 `T` 是一个数组类型，`new` 返回一个指向数组首元素的指针。类似的，`new [n] T` 分配 `n` 个类型为 `T` 的对象，并返回指向数组首元素的指针。

默认情况下，分配的对象进行默认初始化。我们也可以提供可选的初始化器。

定位 new (placement new) 一种 `new` 表达式形式，接受一些额外的参数，在 `new` 关键字后面的括号中给出。例如，`new (nothrow) int` 告诉 `new` 不要抛出异常。

引用计数 (reference count) 一个计数器，记录有多少用户共享一个对象。智能指针用它来判断什么时候释放所指向的对象是安全的。

shared_ptr 提供所有权共享的智能指针：对共享对象来说，当最后一个指向它的 `shared_ptr` 被销毁时会被释放。

智能指针 (smart pointer) 标准库类型，行为类似指针，但可以检查什么时候使用指针是安全的。智能指针类型负责在恰当的时候释放内存。

unique_ptr 提供独享所有权的智能指针：当 `unique_ptr` 被销毁时，它指向的对象被释放。`unique_ptr` 不能直接拷贝或赋值。

weak_ptr 一种智能指针，指向由 `shared_ptr` 管理的对象。在确定是否应释放对象时，`shared_ptr` 并不把 `weak_ptr` 统计在内。

第Ⅲ部分

类设计者的工具

内容

第 13 章	拷贝控制.....	439
第 14 章	操作重载与类型转换.....	489
第 15 章	面向对象程序设计	525
第 16 章	模板与泛型编程.....	577

类是 C++ 的核心概念。我们已经从第 7 章开始详细介绍了如何定义类。第 7 章涵盖了使用类的所有基本知识：类作用域、数据隐藏以及构造函数，还介绍了类的一些重要特性：成员函数、隐式 `this` 指针、友元以及 `const`、`static` 和 `mutable` 成员。在第Ⅲ部分中，我们将延伸类的有关话题的讨论，将介绍拷贝控制、重载运算符、继承和模板。

如前所述，在 C++ 中，我们通过定义构造函数来控制类类型的对象初始化时做什么。类还可以控制在对象拷贝、赋值、移动和销毁时做什么。在这方面，C++ 与其他语言是不同的，其他很多语言都没有给予类设计者控制这些操作的能力。第 13 章将介绍这些内容。本章还会介绍新标准引入的两个重要概念：右值引用和移动操作。

第 14 章介绍运算符重载，这种机制允许内置运算符作用于类类型的运算对象。这样，我们创建的类型直观上就可以像内置类型一样使用，运算符重载是 C++ 借以实现这一目的的方法之一。

类可以重载的运算符中有一种特殊的运算符——函数调用运算符。对于重载了这种运算符的类，我们可以“调用”其对象，就好像它们是函数一样。新标准库中提供了一些设施，使得不同类型的可调用对象可以以一种一致的方式来使用，我们也将介绍这部分内容。

第 14 章最后将介绍另一种特殊类型的类成员函数——转换运算符。这些运算符定义了类类型对象的隐式转换机制。编译器应用这种转换机制的场合与原因都与内置类型转换是一样的。

第Ⅲ部分的最后两章将介绍 C++ 如何支持面向对象编程和泛型编程。

第 15 章介绍继承和动态绑定。继承和动态绑定与数据抽象一起构成了面向对象编程的基础。继承令关联类型的定义更为简单，而动态绑定可以帮助我们编写类型无关的代码，可以忽略具有继承关系的类型之间的差异。

第 16 章介绍函数模板和类模板。模板可以让我们写出类型无关的通用类和函数。新标准引入了一些模板相关的新特性：可变参数模板、模板类型别名以及控制实例化的新方法。

编写我们自己的面向对象的或是泛型的类型需要对 C++ 有深刻的理解。幸运的是，我们无须掌握如何构建面向对象和泛型类型的细节也可以使用它们。例如，标准库中广泛使用了我们将在第 15 章和第 16 章中学习的技术，虽然我们已经使用过标准库类型和算法，但实际上我们并不了解它们是如何实现的。

因此，读者应该明白第Ⅲ部分涉及的是相当深入的内容。编写模板或面向对象的类要求对 C++ 的基本知识和基本类的定义有着深刻的理解。

第 13 章

拷贝控制

内容

13.1 拷贝、赋值与销毁	440
13.2 拷贝控制和资源管理	452
13.3 交换操作	457
13.4 拷贝控制示例	460
13.5 动态内存管理类	464
13.6 对象移动	470
小结	486
术语表	486



如我们在第 7 章所见，每个类都定义了一个新类型和在此类型对象上可执行的操作。在本章中，我们还将学到，类可以定义构造函数，用来控制在创建此类型对象时做什么。



在本章中，我们还将学习类如何控制该类型对象拷贝、赋值、移动或销毁时做什么。类通过一些特殊的成员函数控制这些操作，包括：拷贝构造函数、移动构造函数、拷贝赋值运算符、移动赋值运算符以及析构函数。